Cloud Scalability and Resilience on AWS

What changes when the Product service scales?

Horizontal scaling of the Product-Coupon application

Mark Crowe

MSc in Software Design with Cloud Computing

Container Design and Deployment – CA1

*"~~Peace~~ **Failure** was never an option."* – Magneto, *X-Men: First Class* trailer (2011), adapted

November 2026

Built with Reveal.js, ARJ, Mermaid and ECharts – Source

Download as PDF

© 2026 Mark Crowe

Version 1.9.0

tinyurl.com/3r3c23hm

a00349034.github.io/cloud-scaling-resilience-aws/docs/screencast-deck

About the author



Mark Crowe

Qualifications

- MSc in Software Design with Cloud Computing (2027)
- BSc in Computer Systems

github.com/A00349034

A00349034@outlook.ie

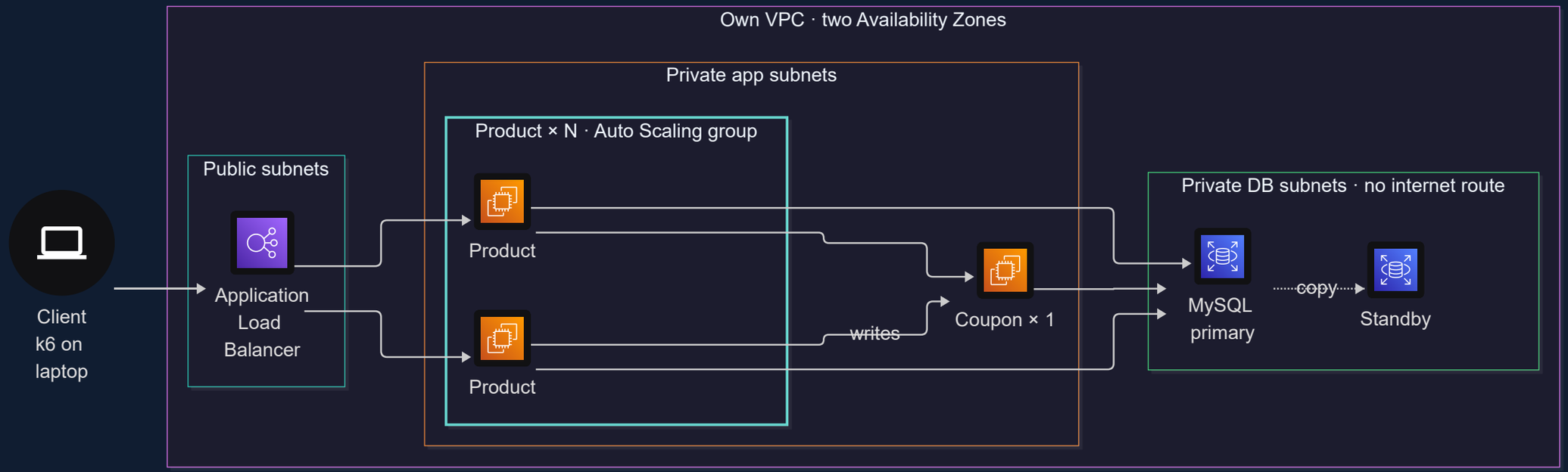
Languages

C++ | C# | Java | JavaScript | Python | TypeScript

Frameworks, data and tools

.NET | .NET MAUI | Big Data | Django | Jupyter Notebooks | MySQL | Reveal.js | Spring

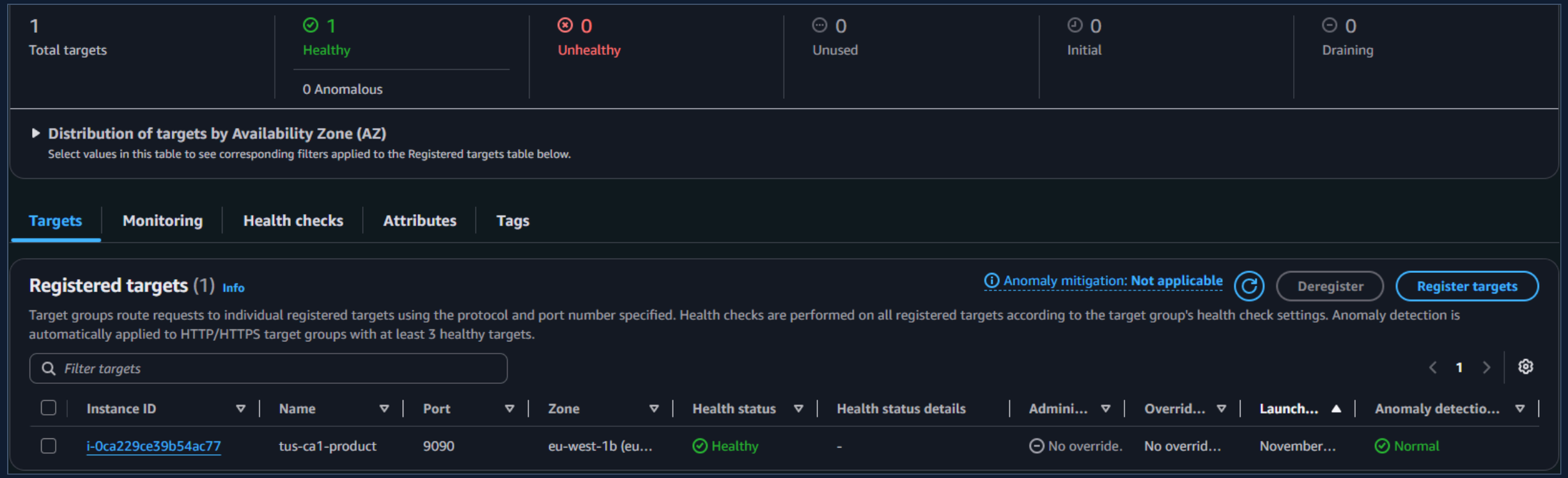
Three tiers, two zones, one public entry



1 – The architecture as built on November 11 (sets A and B). Only the load balancer is public; only the Product tier (outlined) changes

- Changed between tests: the number of Product instances (1, 2, or scaled automatically)
- Single in every test: Coupon. The database has a standby copy in the other zone (Multi-AZ 4)
- First design (set 0, November 9-10): AWS's default network, public instance addresses, no database standby

On the AWS console: one healthy target



2 – The load balancer's target group in the set 0 elasticity run, before scale-out: one Product instance, healthy. Health checks decide where requests go[1]

A controlled comparison, three times

1 Baseline

One Product instance
Scaling off

2 Compare

Two instances
Same workload and data

3 Disrupt

Automatic scaling, lose an instance
Stop Coupon, fail over the database

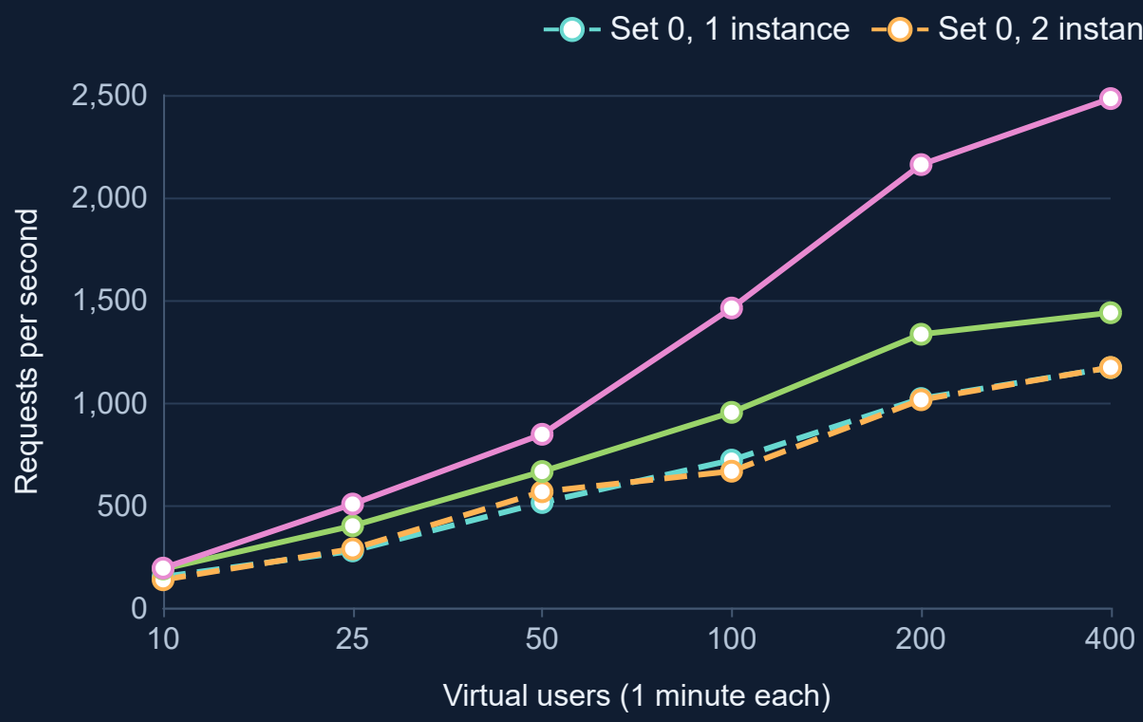
- Three sets: set 0 (first design, November 9-10); set A (new design, standard CPU credits); set B (new design, unlimited CPU credits). A t3.micro earns CPU credits while idle; with standard credits and none left, its CPU is held at 10%[3]
- p95, the 95th percentile[9]: 95% of requests finished faster than this time, measured by k6 (a load-testing tool) on the laptop[6]
- Kept the same: instance type, workload (k6, `GET /productapi/products`), data size (20 products), client location

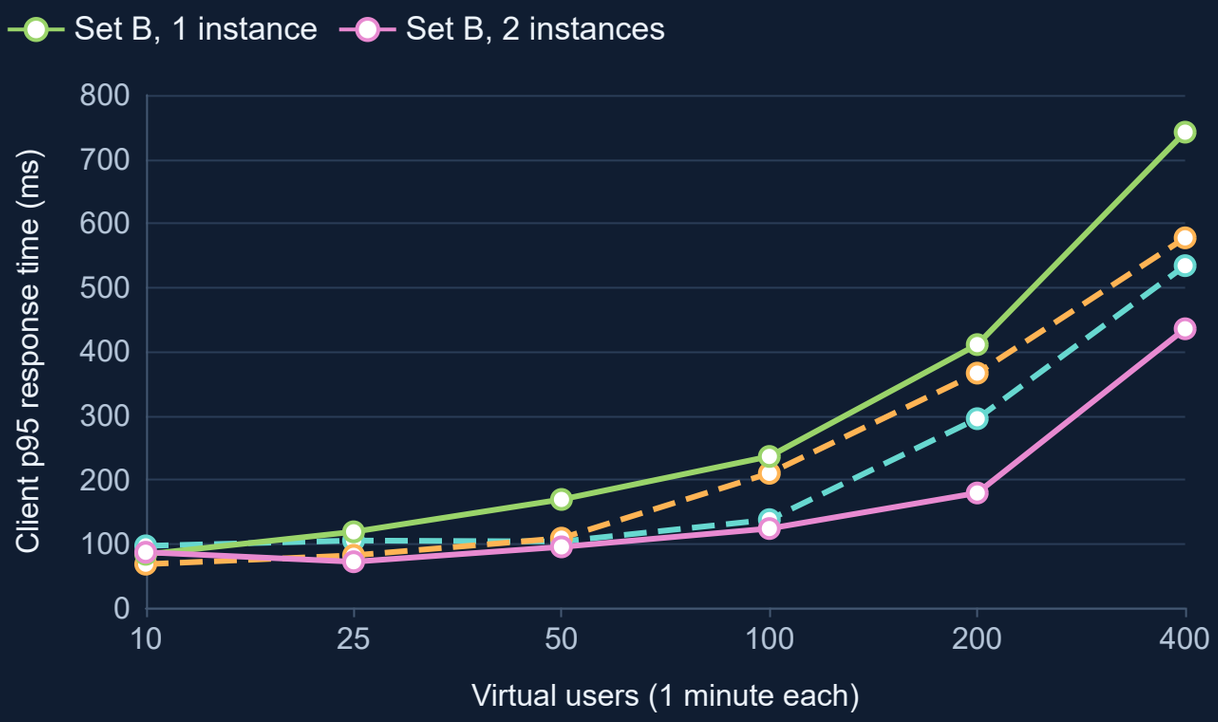
Two instances doubled capacity, once CPU was not capped

At 400 users	Requests per second, 1 instance	Requests per second, 2 instances	Client p95, 1 → 2 instances
Set 0: first design	1,170	1,171	534 → 577 ms
Set A: standard credits, none left	433	861	3,290 → 1,910 ms
Set B: unlimited credits	1,438	2,483	742 → 435 ms

- Set A: CPU held at 10%, so one instance stopped at about 430 requests per second, and two capped instances doubled that[3]
- Set B: two instances served 1.7 times as many requests, and client p95 fell by 41%

Set B reached twice the set 0 ceiling, from the same laptop





3 – `GET /productapi/products` from the laptop, 10 to 400 users, 1 minute each. Dashed: set 0; solid: set B

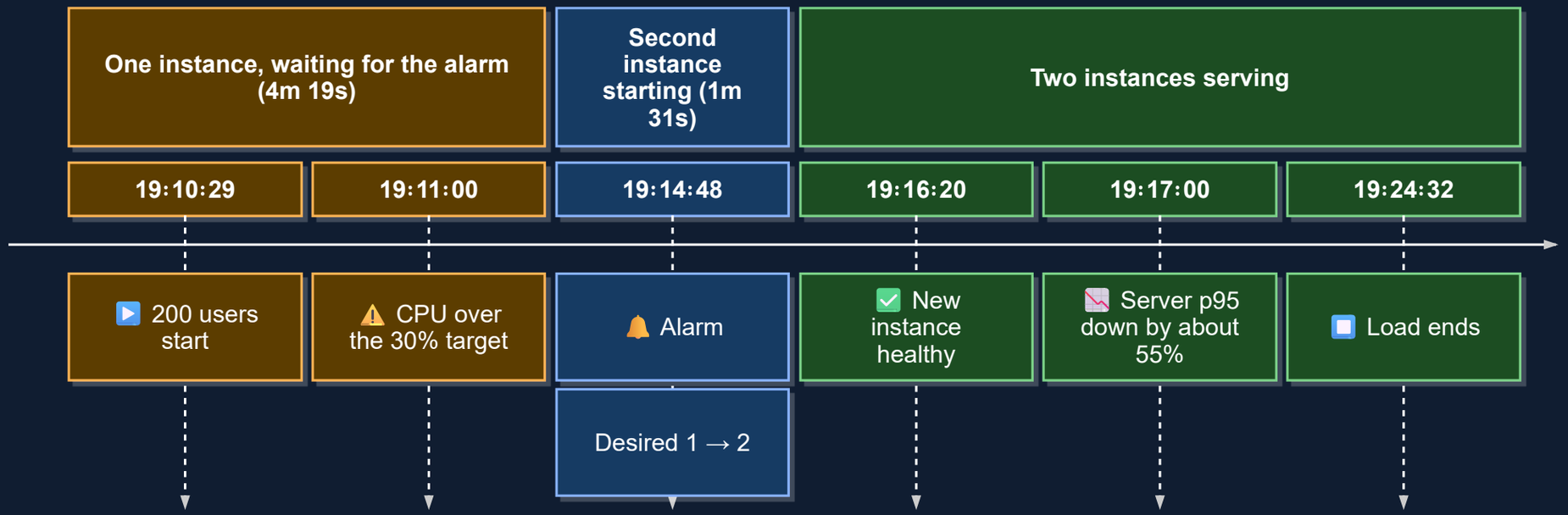
1 – one-vs-two.csv

Scale-out took about 5 minutes, or never happened

From the load increase	Set 0	Set A	Set B
Desired capacity 1 → 2	4m 19s	never	3m 20s
New instance healthy	5m 51s	–	4m 55s
Product CPU, one instance	56-64%	10%, capped	72-76%
Client p95, whole run	361 ms	5,572 ms	231 ms

- Target tracking on average CPU at 30%, minimum 1, maximum 2 instances, 200 users for 14 minutes 2
- Set A never scaled out: CPU was held at 10%, under the 30% target, so the policy saw nothing to act on

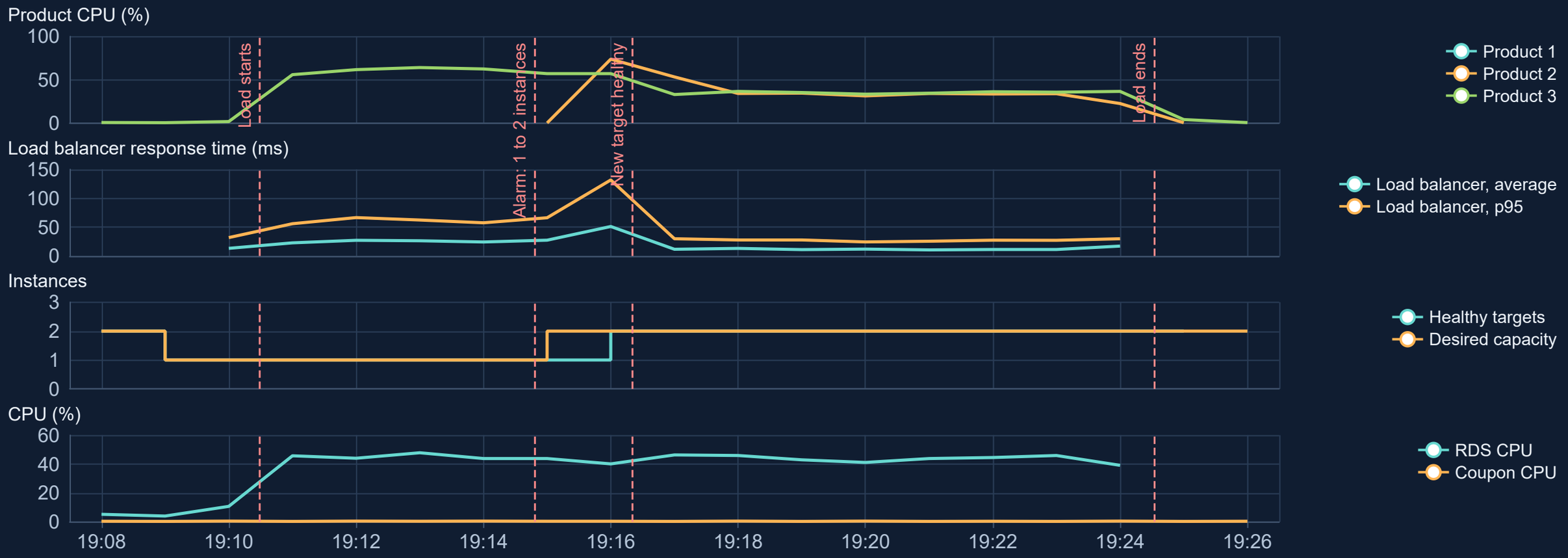
Set 0: the events in order



4 – Elasticity events, set 0 (UTC): 200 users, CPU target 30%[2], min 1, max 2

2 – elasticity-events.csv

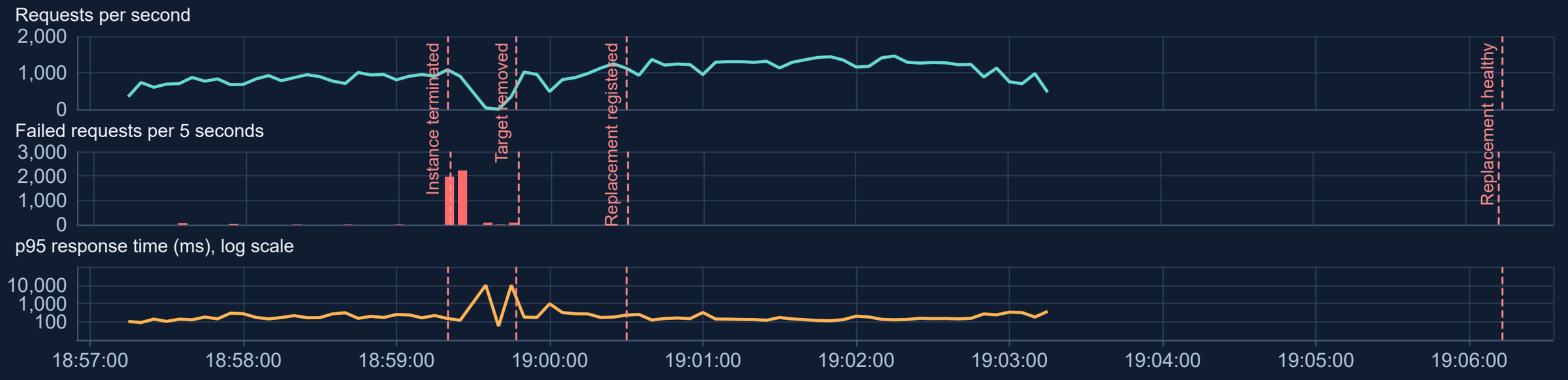
Set 0: what CloudWatch saw



5 – Set 0, 1-minute data from Amazon CloudWatch, the AWS monitoring service

3 – elasticity-cloudwatch.csv

Losing an instance cost 1.3% of requests

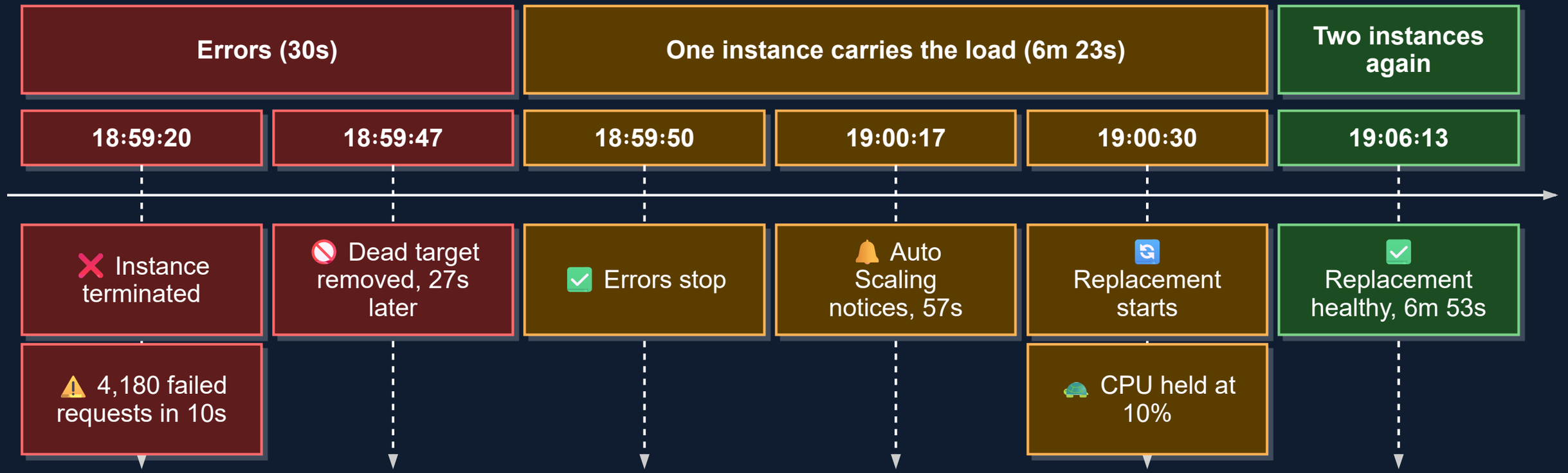


6 – Set 0: 100 users, 2 instances, one terminated at 18:59:20. k6 requests in 5-second windows

4 – failure-client-5s.csv

- Every set: 1.3% of requests failed. The load balancer kept the dead instance for 21-28 seconds, until its health checks failed[1]
- Replacement healthy: 2m 15s with unlimited credits; about 7 minutes with standard credits (CPU held at 10%)[3]

Set 0: the events in order



🖼 7 – Failure events, set 0 (UTC): the client recovered in seconds, full capacity took almost 7 minutes

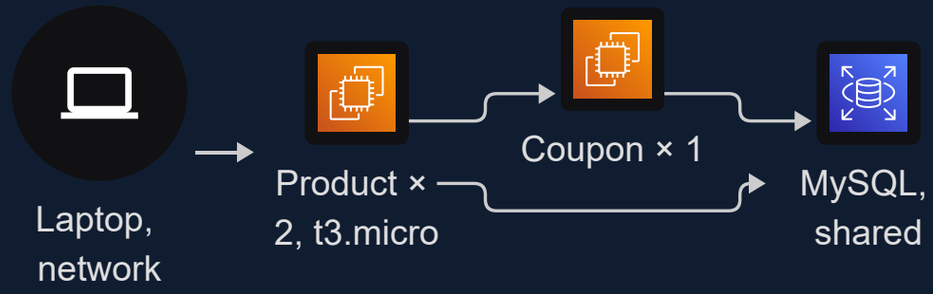
CSV 5 – CSV failure-events.csv

When a shared dependency fails

	Coupon stopped (H06), sets 0, A, B	Database failover (H07), sets A, B
Reads	0% failed	0-0.01% failed
Writes	20 of 20 failed, in every set	1 failed; working again after 20 s
Back to normal	13-18 s after Coupon restarted	Standby took over in 40-59 s
Product instance	Stayed healthy	Stayed healthy

- Coupon: Product's health check did not notice, so the load balancer kept sending writes to it. More Product instances cannot help
- Database: Multi-AZ switched to the standby, with the same address[4]; Product reconnected without a restart

What remains limited?



- CPU credits: with none left, one instance served 433 requests per second instead of 1,438, and scaling never started
- Database: shared by every instance; its CPU rose from 41% to 70% with two instances
- Coupon: one instance on the write path; stopped, every write failed
- Start-up: about 2 minutes with unlimited credits, about 7 with standard credits

Above: what every Product instance depends on. A single dependency is a risk; it is only a bottleneck where the data shows saturation.

Answer

Scalability

With unlimited credits, two instances served 1.7 times as many requests and p95 fell 41%. With credits used up, CPU was capped and scaling never started

Resilience

Losing an instance cost 1.3% of requests; a database failover cost one write. A stopped Coupon broke every write

Next improvement

Minimum 2 Product from a pre-built image, unlimited credits, Coupon in each zone, a paged product list; then run k6 inside AWS

Already built: private subnets and a Multi-AZ database, at about USD 32-35 a month for each NAT gateway[5]. Details: report, section 5.4

Thanks for coming to my TED Talk

🎤 Questions? ✉ Email me

A00349034@outlook.ie

- The full report: MD report.md · PDF Download the report as a PDF
- References, figures, software and links are below ↓, and → to exit

References

1. Amazon Web Services. Health checks for Application Load Balancer target groups. Elastic Load Balancing User Guide
2. Amazon Web Services. Target tracking scaling policies for Amazon EC2 Auto Scaling. Amazon EC2 Auto Scaling User Guide
3. Amazon Web Services. Unlimited mode for burstable performance instances. Amazon EC2 User Guide
4. Amazon Web Services. Multi-AZ DB instance deployments for Amazon RDS. Amazon RDS User Guide
5. Amazon Web Services. Amazon VPC pricing (NAT gateway)
6. Grafana Labs. k6 documentation: metrics (http_req_duration, p95)
7. Mermaid. Diagrams and charts from text (version 12.3)
8. Apache ECharts. Interactive charts in the browser (version 5.5)
9. Amazon Web Services. CloudWatch statistics definitions: percentiles (p95). Amazon CloudWatch User Guide

Figures

1. The architecture as built: three tiers across two zones
2. The load balancer's target group on the AWS console (set 0)
3. One versus two Product instances, sets 0 and B: requests per second and client p95 against users
4. Elasticity, set 0: the events in order
5. Elasticity, set 0: Product CPU, response time, capacity and RDS CPU
6. Failure, set 0: client requests per second, failed requests and p95
7. Failure, set 0: the events in order

The charts are drawn in the browser from ca1-charts.json, made by ca1_chart_data.py from the saved data in test-results. The data behind each figure: Data sets; the notebook make_charts.ipynb shows the numbers and draws the report's charts.

Data sets

Dataset	File	Description	Data rows	Figure		
1	one-vs-two.csv	One Product instance versus two, sets 0 and B: requests per second and client p95 by users	24	3	View	Download
2	elasticity-events.csv	Elasticity: the events in order	6	4	View	Download
3	elasticity-cloudwatch.csv	Elasticity: CloudWatch, 1-minute points	134	5	View	Download
4	failure-client-5s.csv	Failure: k6 requests in 5-second windows	72	6	View	Download
5	failure-events.csv	Failure: the events in order	6	7	View	Download

View: the data as a table, with what each column means. Download: the CSV file. Made by csv_markdown.py; all data sets: readme.md

Report: report.md · Slides: slides.md · Code: scripts · Evidence: test-results

Software used

Applications

- AWS CLI v2
- Docker
- Java 17 (Amazon Corretto)
- Spring Boot 4.1

AWS

- Application Load Balancer
- CloudWatch
- EC2
- EC2 Auto Scaling
- RDS for MySQL
- Systems Manager

Data and charts

- Jupyter
- matplotlib
- pandas
- Python

Diagrams and slides

- Apache ECharts 5.5
- ARJ 1.8
- Mermaid 12.3
- Reveal.js 5.1

Load testing

- Grafana k6

Scripts

- PowerShell 7

Video

- ffmpeg
- Puppeteer
- Python

Links

- Report: report.md · Download the report as a PDF
- These slides: slides.md, narration speaking-script.md · Download the slides as a PDF
- Code and evidence: github.com/A00349034/cloud-scaling-resilience-aws
- Notes: © 2026 Mark Crowe. AWS service and software icons from Iconify's "logos" pack (CC0) and Devicon (MIT); other icons Material Design Icons (Apache 2.0); site icons belong to their sites; reveal.js © Hakim El Hattab, MIT licence. Generative AI (Claude) helped build the scripts, diagrams and slides; all measurements are from my own AWS deployment



That's all, folks !